CO3 – AT1

CASE STUDY BASED ASSESSMENT

REPORT:

Experiment: Quick Sort for a Real-Time Ranking System

Aim

To implement the **Quick Sort** algorithm for dynamically sorting player scores in a real-time leaderboard and analyze its performance using the Divide and Conquer technique.

Algorithm

1. Start.
2. Read the array of player scores.
3. Choose a pivot element.
4. Partition the array so that larger scores are on the left and smaller scores are on the right.
5. Recursively apply Quick Sort to the left sub-array.
6. Recursively apply Quick Sort to the right sub-array.
7. Repeat until all elements are sorted.
8. Display the sorted leaderboard.
9. Stop.

Source code and Output Screenshot:

main.c

```c
#include <stdio.h>
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
int partition(int arr[], int low, int high)
{
    int pivot = arr[high]; // Last element as pivot
    int i = low - 1;
    for(int j = low; j < high; j++)
    {
        if(arr[j] > pivot) // Descending order for leaderboard
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}
void quickSort(int arr[], int low, int high)
{
    if(low < high)
    {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
```

Output

```
Original Leaderboard Scores:
450 120 800 350 620 980 500

Sorted Leaderboard (Highest Score First):
980 800 620 500 450 350 120

=== Code Execution Successful ===
```

main.c

```c
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}
void quickSort(int arr[], int low, int high)
{
    if(low < high)
    {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
int main()
{
    int scores[] = {450, 120, 800, 350, 620, 980, 500};
    int n = sizeof(scores) / sizeof(scores[0]);
    printf("Original Leaderboard Scores:\n");
    for(int i = 0; i < n; i++)
        printf("%d ", scores[i]);
    quickSort(scores, 0, n - 1);
    printf("\n\nSorted Leaderboard (Highest Score First):\n");
    for(int i = 0; i < n; i++)
        printf("%d ", scores[i]);
    return 0;
}
```

Output

```
Original Leaderboard Scores:
450 120 800 350 620 980 500

Sorted Leaderboard (Highest Score First):
980 800 620 500 450 350 120

=== Code Execution Successful ===
```

Sample Output

Original Leaderboard Scores:

450 120 800 350 620 980 500

Sorted Leaderboard (Highest Score First):

980 800 620 500 450 350 120

Working Principle (Divide and Conquer)

Quick Sort follows three steps:

1. **Divide**
    - Select a pivot element.
    - Partition the array so that larger scores are placed before the pivot (for descending order).
2. **Conquer**
    - Recursively apply Quick Sort to the left and right partitions.
3. **Combine**
    - Since partitioning places the pivot in its correct position, no extra merge operation is required.

Issues in Quick Sort

1. Pivot Selection

Choosing the first or last element as the pivot may produce unbalanced partitions when the data is already sorted or nearly sorted.

2. Worst-Case Performance

If the pivot consistently becomes the smallest or largest element, the recursion depth increases significantly.

Example:

10 20 30 40 50

Selecting the last element repeatedly creates one empty partition and one partition of size $n - 1$.

Time Complexity:

$O(n^2)$

Randomized Pivot Selection

Instead of always selecting the last element, choose a random pivot.

Advantages

- Reduces the probability of worst-case partitions.
- Produces more balanced recursive calls.
- Makes average performance close to O(n log n) even for nearly sorted data.
- Improves performance in real-time leaderboard applications where scores change frequently.

Time Complexity

Case	Time Complexity
Best Case	**O(n log n)**
Average Case	**O(n log n)**
Worst Case	**O(n²)**

Space Complexity

Case	Space Complexity
Best/Average	**O(log n)** (recursive stack)
Worst	**O(n)**

Result

The Quick Sort algorithm was successfully implemented to sort leaderboard scores in descending order. By applying the Divide and Conquer approach, it efficiently organized the rankings. Although poor pivot selection can lead to $O(n^2)$ performance, using a randomized pivot significantly improves efficiency, making Quick Sort well suited for real-time ranking systems.